Practical Deep Reinforcement Learning Approach for Stock Trading Implementation

Project Report
Xiaoyan Lin, Zhuoqiao Ouyang

May 2025

1 Introduction

In this project the we study and attempt to replicate a stock trading strategy with Deep Reinforcement Learning approach proposed by Liu et al [3]. The author mentioned two approaches for seeking profitable trading strategies, one through the traditional method of optimizing portfolio weights and the other through modeling the stock trading problem as a Markov Decision Process (MDP) and apply Dynamic Programming techniques. The traditional techniques such as value iteration are often computationally inapplicable when state and action spaces are large and deep reinforcement learning techniques are used for approximating the solutions. Liu et al explored the Deep Deterministic Policy Gradient algorithm (DDPG), which uses the actor-critic framework to model large state and action spaces, target network to stabilize the training process, and experience replay for correlation reduction between samples, and claims that the DDPG algorithm over performs the min-variance portfolio and DJIA. In this report, we will give a general overview of the algorithm, then we will explain our attempt in replicating the strategy, finally we present the obtained numerical results.

2 An Overview of the Deep Deterministic Policy Gradient (DDPG) Algorithm

2.1 Trading Problem as a Markov Decision Process

To reflect the stochastic and interactive nature of financial markets, we model the stock trading process as a Markov Decision Process (MDP). Let $\mathbb{R}$ denote the set of real numbers, let $\mathbb{R}_+ = \{x \in \mathbb{R} : x \geq 0\}$ denote the set of nonnegative real numbers. Let d be the number of stocks in the market. The agent observes the state

$$s = [p, h, b],$$

where $p \in \mathbb{R}_+^d$ denotes the vector of stock prices, $h \in \mathbb{R}_+^d$ denotes the vector of stock holdings, and $b \in \mathbb{R}_+$ denotes the remaining cash balance. Since the state consists of d stock prices, d stock holdings, and one cash-balance variable, the total dimension of the state space is

$$\dim(s) = 2d + 1.$$

The actor network outputs a continuous action vector

$$a = (a_1, \ldots, a_d) \in [-1, 1]^d,$$

where each component a_i represents the trading signal for stock i. Positive values indicate buying, negative values indicate selling, and zero indicates holding. On the sell side, the environment clips actions so that the agent cannot sell more than it currently holds:

$$a \leftarrow \max(a, -h).$$

On the buy side, the environment checks whether the current cash balance is sufficient to pay for the purchase. If the cost of a proposed buy exceeds the remaining balance, that component is not executed.

The reward function $r(s, a, s')$ is defined as the change in portfolio value when action a is taken at state s and the environment transitions to the next state s'. The portfolio value is given by

$$v = p^\top h + b,$$

which is the sum of the market value of all held stocks and the remaining cash balance. Accordingly, the immediate reward is measured by the change in portfolio value between two consecutive time steps. The policy, denoted by $\pi(s)$, represents the trading strategy followed by the agent at state s.

The action-value function, denoted by $Q^\pi(s, a)$, represents the expected cumulative reward obtained by taking action a at state s and then following policy π thereafter.

At each time step t, where each time step corresponds to one trading day, the Bellman equation implies that the action-value function at state s_t and action a_t is equal to the expected immediate reward plus the discounted expected future reward obtained from the next state s_{t+1} [3]. Therefore,

$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}}\left[r(s_t, a_t, s_{t+1}) + \gamma\, \mathbb{E}_{a_{t+1}\sim\pi(s_{t+1})}\left[Q_\pi(s_{t+1}, a_{t+1})\right]\right] \tag{1}$$

where γ is the discount factor.
The objective is to design a trading strategy that maximizes the terminal portfolio value at a target time t_f, that is,

$$p_{t_f}^\top h_{t_f} + b_{t_f},$$

which is equivalent to maximizing the cumulative reward $\sum_{t=1}^{t_f-1} r(s_t, a_t, s_{t+1})$. By the Markov property of the MDP formulation, this reduces to finding a policy that maximizes the action-value function $Q_\pi(s_t, a_t)$. This problem is particularly challenging because the action-value function is unknown and must be learned through repeated interaction with the market environment, where at each step the agent observes the state and outputs a continuous action vector. To address this, we employ a deep reinforcement learning approach to solve this problem.

Revision of Subsection: Trading problem as a Markov decision process

The portfolio trading problem can be naturally formulated as a Markov Decision Process. Suppose our initial cash for investment is $C \in \mathbb{R}^+$, trading period is $T > 0$ and the portfolio contains n assets. Then At each time step t of trades and for each asset i, we observe its the price p_{ti}, the holdings/position h_{ti} and the remaining cash b_t. The actions we are allowed to take is buying or selling $k > 0$ (in this problem we do not consider short selling) shares or do nothing. Let $\mathbb{X}$ and $\mathbb{A}$ denotes the state and action spaces, then at each time t a state can be written as

$$x_t = (p_1, \ldots, p_n, h_1, \ldots, h_n, b) \in \mathbb{R}_+^n \times \mathbb{Z}_+^n \times \mathbb{R}_+;$$
$$a_t = (k_1, \ldots, k_n) \in A(x_t),$$

where $A(x_t)$ is the set of the all feasible actions at the state x_t such that the remaining cash is nonnegative at state x_{t+1}. The reward function at the state-action pair $r(x_t, a_t, x_{t+1})$ is then the profits or loss of taking action a_t at state x_t and reaching state x_{t+1}. The portfolio value at time t is then

$$v = (p_1, \ldots, p_n)^\top (h_1, \ldots, h_n) + b.$$

The objective is to find an optimal deterministic policy $\pi \in \Pi, \pi(x_t) = a_t$ that maximizes the portfolio value at a target time T in the future, equivalently maximizing the total rewards $\sum_{t=1}^{T} r(x_t, a_t, x_{t+1})$. The expected reward of taking action a_t at state x_t is the current reward plus the discounted expected reward at the next state:

$$V^\pi(x_{t+1}) = \mathbb{E}_{a_{t+1}\sim\pi(x_{t+1})}\left[Q^\pi(x_{t+1}, a_{t+1})\right].$$
$$Q^\pi(x_t, a_t) = \mathbb{E}_\pi\left[r(x_t, a_t, x_{t+1}) + \gamma V^\pi(x_{t+1}) \mid x_t, a_t\right].$$

where γ is the discount factor. The optimization problem is then equivalent to maximizing $Q^\pi(x_t, a_t)$.

3 Deep Deterministic Policy Gradient (DDPG)

DDPG adopts an actor–critic architecture, in which two networks play complementary roles. The *actor* network $\mu(s|\theta^\mu)$ is responsible for learning the trading policy by mapping states to deterministic actions, where θ^μ denotes the parameters of the actor network. The *critic* network $Q(s, a|\theta^Q)$ is responsible for evaluating the quality of the actions taken by the actor by approximating the action-value function $Q_\pi(s, a)$, where θ^Q denotes the parameters of the critic network.

Q-learning is essentially a value-based method for learning the action-value function of the environment. Instead of taking the expectation over the next action a_{t+1}, classical Q-learning uses the greedy action that maximizes the action-value function at the next state s_{t+1}, that is,

$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}}\left[r(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})\right]. \tag{2}$$

Although DDPG is not a standard Q-learning algorithm, it still incorporates the idea of action-value learning through the critic network. In particular, the critic approximates the action-value function $Q(s, a)$ and is updated using a temporal-difference target. However, unlike classical Q-learning, DDPG does not explicitly compute $\max_{a'} Q(s', a')$ over all possible next actions. Instead, it uses a target actor network to generate the next action and a target critic network to evaluate that action.

3.1 Network Architecture

Figure 1 illustrates the actor–critic architecture of the DDPG model used in this project. The actor network, denoted by $\mu(s|\theta^\mu)$, takes the current state s as input and outputs an action a. The output action corresponds to the trading decision of the agent. An exploration noise term $\mathcal{N}$ is added to the actor output for better action exploration. The critic network, denoted by $Q(s, a|\theta^Q)$, then receives both the current state and the selected action as inputs and outputs the value of the action under that state. The value of action under that state evaluates the quality of taking action a under state s.

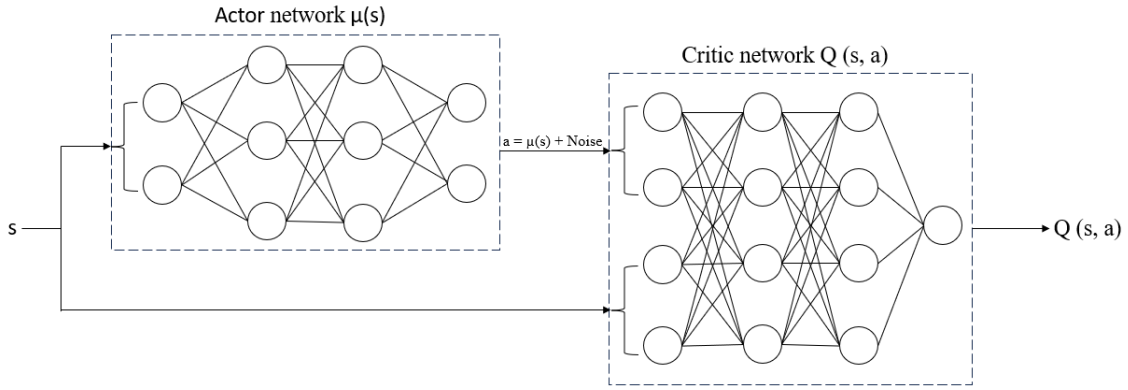


Figure 1: Learning network architecture

3.2 Replay Buffer

To stabilize training, DDPG employs a *replay buffer* $\mathcal{R}$, which stores the transition tuple

$$(s_t,\ a_t,\ s_{t+1},\ r_t)$$

at each time step t, after the agent takes action a_t at state s_t and transitions to the next state s_{t+1}. A mini-batch of N transitions

$$\{(s_i,\ a_i,\ r_i,\ s_i + 1)\}_{i=1}^N$$

is drawn at random from $\mathcal{R}$ to update the actor and critic networks. This random sampling effectively reduces the correlation between experience samples.

3.3 Update Actor, Critic and Target Networks

The actor policy is updated by using the sampled policy gradient, while the critic is updated by minimizing the loss. The N sample transitions are drawn from $\mathcal{R}$ and the Bellman target is computed as

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} \mid \theta^{\mu'}) \mid \theta^{Q'}), \quad i = 1, \ldots, N,$$

where μ' and Q' are the target actor and target critic networks, created by copying the actor and critic networks respectively, so that they provide consistent temporal difference backups. The critic network is then updated by minimizing the expected difference $L(\theta^Q)$ defined as the expected difference between the outputs of the target critic network Q' and the critic network Q, i.e.,

$$L(\theta^Q) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1} \sim \text{buffer}} \left[\left(r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1} \mid \theta^{\mu'}) \mid \theta^{Q'}) - Q(s_t, a_t \mid \theta^Q) \right)^2 \right]. \tag{3}$$

Since a finite number of N sample transitions are drawn from the replay buffer $\mathcal{R}$, the expectation can be approximated by the empirical average, and the loss function 10 can be written as

$$L(\theta^Q) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - Q(s_i, a_i \mid \theta^Q) \right)^2. \tag{4}$$

The parameters θ^μ of the actor network are then updated using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t, a_t, r_t, s_{t+1} \sim \text{buffer}} \left[\nabla_{\theta^\mu} Q(s_t, \mu(s_t \mid \theta^\mu) \mid \theta^Q) \right] \tag{5}$$

$$= \mathbb{E}_{s_t, a_t, r_t, s_{t+1} \sim \text{buffer}} \left[\nabla_a Q(s_t, \mu(s_t) \mid \theta^Q) \nabla_{\theta^\mu} \mu(s_t \mid \theta^\mu) \right]. \tag{6}$$

Since a finite number of N sample transitions are drawn from the replay buffer $\mathcal{R}$, the expectation can be approximated by the empirical average, and the policy gradient can be written as

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^{N} \left[\nabla_a Q(s_i, a \mid \theta^Q) \big|_{a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s_i \mid \theta^\mu) \right]. \tag{7}$$

Following the update of the actor and critic networks using transitions sampled from the replay buffer, the target actor and target critic networks are updated according to:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}, \tag{8}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}, \tag{9}$$

where $\tau \ll 1$ is the soft update rate, ensuring that the target networks track the learned networks slowly for stable training.
The following algorithm outlines the training procedure of the Deep Deterministic Policy Gradient (DDPG) model for portfolio management:

Algorithm 1 DDPG training algorithm for portfolio management

Initialize actor network $\mu(s \mid \theta^{\mu})$ and critic network $Q(s, a \mid \theta^{Q})$ with random parameters
Initialize target networks μ' and Q' with parameters $\theta^{\mu'} \leftarrow \theta^{\mu}$ and $\theta^{Q'} \leftarrow \theta^{Q}$
Initialize replay buffer $\mathcal{R}$
Initialize environment with training data and initial portfolio value
for episode $= 1, \ldots, M$ **do**
    Reset environment and receive initial state s_t
    Set `done` $\leftarrow$ `False`
    Initialize portfolio value history:

$$\mathcal{V} \leftarrow [V_0],$$

where V_0 is the initial portfolio value
    while not done **do**
        Compute action from the actor network:

$$a_t = \mu(s_t \mid \theta^{\mu}) + \epsilon_t,$$

where $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$
        Execute action a_t in the environment
        Observe next state s_{t+1}, reward r_t, and terminal indicator d_t
        Record the updated portfolio value V_{t+1} and append it to $\mathcal{V}$
        Store transition $(s_t, a_t, r_t, s_{t+1}, d_t)$ in replay buffer $\mathcal{R}$
        Set $s_t \leftarrow s_{t+1}$
        if $|\mathcal{R}| \geq N$ **then**
            Sample a minibatch $\{(s_i, a_i, r_i, s_{i+1}, d_i)\}_{i=1}^{N}$ from $\mathcal{R}$
            Compute target actions:

$$a'_{i+1} = \mu'(s_{i+1} \mid \theta^{\mu'})$$

            Compute target values:

$$y_i = r_i + \gamma(1 - d_i)\, Q'(s_{i+1}, \mu'(s_{i+1} \mid \theta^{\mu'}) \mid \theta^{Q'})$$

            Update critic parameters by minimizing

$$L(\theta^{Q}) = \frac{1}{N} \sum_{i=1}^{N} \big(Q(s_i, a_i \mid \theta^{Q}) - y_i\big)^2$$

            Update actor parameters by minimizing

$$L_{\text{actor}}(\theta^{\mu}) = -\frac{1}{N} \sum_{i=1}^{N} Q\big(s_i, \mu(s_i \mid \theta^{\mu}) \mid \theta^{Q}\big)$$

            Soft-update the target critic network and actor network:

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1 - \tau)\theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1 - \tau)\theta^{\mu'}$$

        end if
    end while
end for
return trained actor network and portfolio value history $\mathcal{V}$

Revision of Section 5: The DDPG Problem

1. The DDPG algorithm in Liu's paper is specified for the MDP for stock trading. In general, the DDPG algorithm combines the frame work of policy gradient with neural networks as function approximator and Q-learning, which uses greedy action a_{t+1} that maximizes $Q(x_{t+1}, a_{t+1})$ for state x_{t+1}. To be more specific:

2. The actor policy is updated by using the sampled policy gradient, while the critic is updated by minimizing the loss. Suppose at time t, the N sample transitions are drawn from $\mathcal{R}$. Define

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} \mid \theta^{\mu'}) \mid \theta^{Q'}), \quad i = 1, \ldots, N,$$

where μ' and Q' are the target actor and target critic networks, created by copying the actor and critic networks respectively. The critic network is then updated by minimizing the expected difference $L(\theta^Q)$ defined as the expected difference between the outputs of the target critic network Q' and the critic network Q, i.e.,

$$L(\theta^Q) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1} \sim \text{buffer}} \left[\left(y_i - Q(s_t, a_t \mid \theta^Q) \right)^2 \right]. \tag{10}$$

Since a finite number of N sample transitions are drawn from the replay buffer $\mathcal{R}$, the expectation can be approximated by the empirical average, and the loss function 10 can be written as

$$L(\theta^Q) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - Q(s_i, a_i \mid \theta^Q) \right)^2. \tag{11}$$

The parameters θ^μ of the actor network are then updated using the sampled policy gradient, which can be rewritten as

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^{N} \left[\nabla_a Q(s_i, a \mid \theta^Q) \big|_{a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s_i \mid \theta^\mu) \right]. \tag{12}$$

Following the update of the actor and critic networks using transitions sampled from the replay buffer, the target actor and target critic networks are updated according to:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}, \tag{13}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}, \tag{14}$$

where $\tau \ll 1$ is the soft update rate, ensuring that the target networks track the learned networks slowly for stable training.

4 Case Study

For the case study we replicate the DDPG method and compare with the numerical results in Liu's paper, thus we will use the same experimental settings as Liu whenever they are specified in the paper.

4.1 Numerical experiment setting

The data used are the daily adjusted prices of the 30 stocks of the Dow Jones Industrial Average (DJIA) over the period from January 2, 2009 to September 28, 2018. The tickers included in the dataset are MMM, AMZN, AXP, AMGN, AAPL, BA, CAT, CVX, CSCO, KO, DIS, GS, HD, HON, IBM, JNJ, JPM, MCD, MRK, MSFT, NKE, NVDA, PG, CRM, SHW, TRV, UNH, VZ, V, and WMT. The initial cash for the investment is 10000 and transaction cost is assumed to be 0. In the original paper, the data are divided into training set, validation set and testing set. The training set spans from January 2009 to January 2015 and is used for model training, the validations set is the

whole year of 2015, while the test set covers the period from January 2016 to September 2018 and is used for out-of-sample evaluation of all strategies. Validation set is often used to check whether results are theoretically accurate before passing to unseen data, however under the context of stock trading there is no existing theory for comparison, we combine the validation set into the training set.

There are multiple parameters for the actor and critic networks setup:

- The number of layers, dimensions and the choices of activation function;
- discount rate γ and learning rate τ;
- size of the minibatch;
- number of episodes;
- the process which the random noise follows.

None of these parameters are specified in the paper, thus we choose the classical setups. Each network has three layers with 64 sigmoids and all activation functions are linear. The minibatch size is set to $N = 64$, and the exploration noise follows a Gaussian process with standard deviation is $\sigma = 1$ and mean 0. The actor and critic networks are trained using learning rates of 10^{-4} and 10^{-3}, respectively. The replay buffer capacity is set to 100000 transitions. We attempted to tune the parameters for γ and τ in the discretized ranges of $[0, 9, 1)$ and $[0.001, 0.01]$ using 100 episodes for 10 times; while the outputs are unstable, we do noticed that $\gamma = 0.99, \tau = 0.001$ always lead to validation results that outperform other combinations. This observation also match the parameters specified in the paper that introduces the DDPG algorithm. Thus we fix $\gamma = 0.99, \tau = 0.001$ and train the model using 500 episodes and 1000 episodes.

Portfolio performance is evaluated using several summary statistics computed from the portfolio value series. The initial portfolio value is recorded as the starting wealth, and the final portfolio value measures the terminal wealth at the end of the investment horizon. Daily returns are computed as

$$r_t = \frac{V_t - V_{t-1}}{V_{t-1}}, \tag{15}$$

where V_t denotes the portfolio value at time t. The annualized return is calculated as the mean daily return multiplied by 252, while the annualized volatility is computed as the standard deviation of daily returns multiplied by $\sqrt{252}$. The Sharpe ratio is then defined as

$$\frac{r_{\text{ann}} - r_f}{\sigma_{\text{ann}}}, \tag{16}$$

where r_f is the risk-free rate. The risk free rate is also not mentioned in the paper and we assume it is equal to 1.5%; and in fact our computation of the min-variance portfolio using this r_f gives the same result as in the paper. To evaluate downside risk, we also compute the drawdown series, defined as the percentage decline of the portfolio value from its running peak. Based on this series, we report both the maximum drawdown, which captures the largest peak-to-trough loss over the sample period, and the average drawdown, which measures the mean drawdown level across time.

5 Result

5.1 In sample result

Table 1 presents the in-sample performance comparison. The results show that the DDPG strategy outperforms the other two benchmark strategies across the main performance measures.

Table 1: In Sample Performance Comparison

	DDPG (ours)	Mean-Variance	DJIA
Initial Portfolio Value	10000.000000	10000.000000	10000.000000
Final Portfolio Value	27295.618002	19750.795177	19286.802832
Annualized Return	0.160409	0.106599	0.107431
Annualized Std. Error	0.182687	0.119541	0.163780
Sharpe Ratio	0.795947	0.766255	0.564361
Max Drawdown	0.216857	0.186764	0.275343
AVG Drawdown	0.030745	0.028970	0.034735
Sortino Ratio	329.990336	321.033292	229.478546

The results demonstrate that the DDPG-based strategy significantly outperforms both the Min-Variance portfolio and the DJIA benchmark across most performance metrics (See Figure 2). Starting from the same initial capital, the DDPG portfolio achieves a substantially higher final portfolio value, nearly doubling the performance of the benchmark strategies. This superior performance is further supported by its higher annualized return and Sharpe ratio, indicating that the DDPG strategy not only generates higher returns but also achieves better risk-adjusted performance. Although the maximum drawdown of DDPG is comparable to the Min-Variance strategy, it remains significantly lower than that of DJIA, suggesting improved downside risk control. Additionally, the DDPG strategy exhibits the lowest average drawdown, reflecting more stable performance over time. The notably higher Sortino ratio further highlights its effectiveness in minimizing downside volatility. Overall, these results are consistent with findings in prior literature, where deep reinforcement learning approaches such as DDPG have been shown to outperform traditional portfolio allocation methods in both return and risk management.

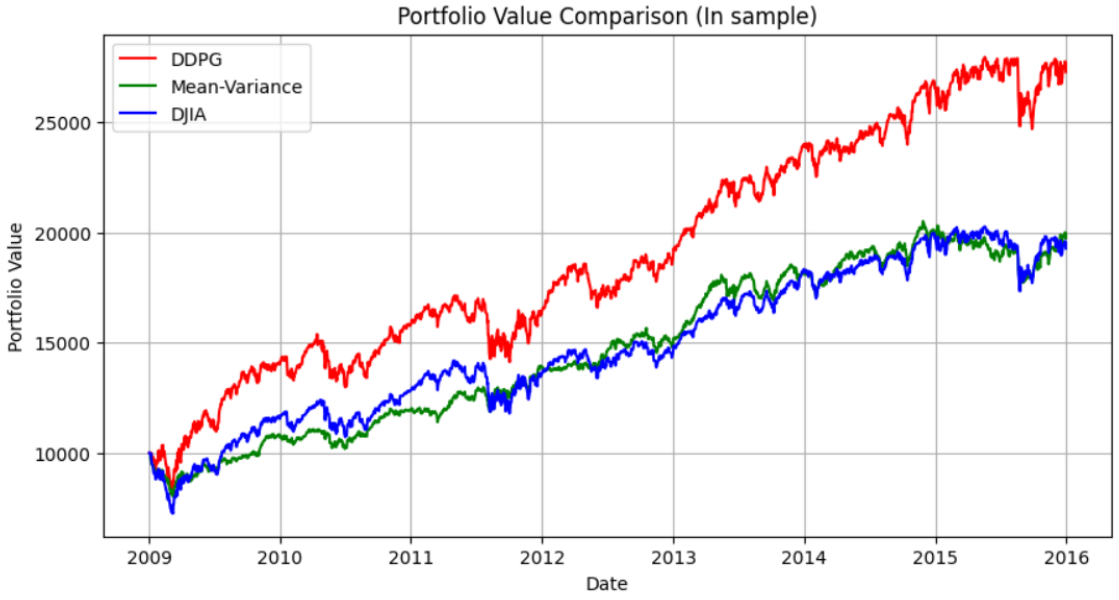


Figure 2: In sample portfolio value curves of our DDPG scheme, the mean-variance portfolio allocation strategy, and the Dow Jones Industrial Average (DJIA), with an initial portfolio value of $10,000.

Figure 3 illustrates the drawdown dynamics of the DDPG strategy, the mean-variance portfolio, and the DJIA benchmark for the in-sample periods. All three strategies experience periods of

significant drawdowns, particularly during market stress episodes. The DJIA and DDPG strategies exhibit relatively higher and more frequent spikes in drawdown, indicating greater exposure to market volatility. In contrast, the mean-variance portfolio generally maintains lower drawdown levels, suggesting more stable downside risk control. However, the DDPG strategy is able to recover more quickly from drawdown events in several periods, reflecting its adaptive nature.

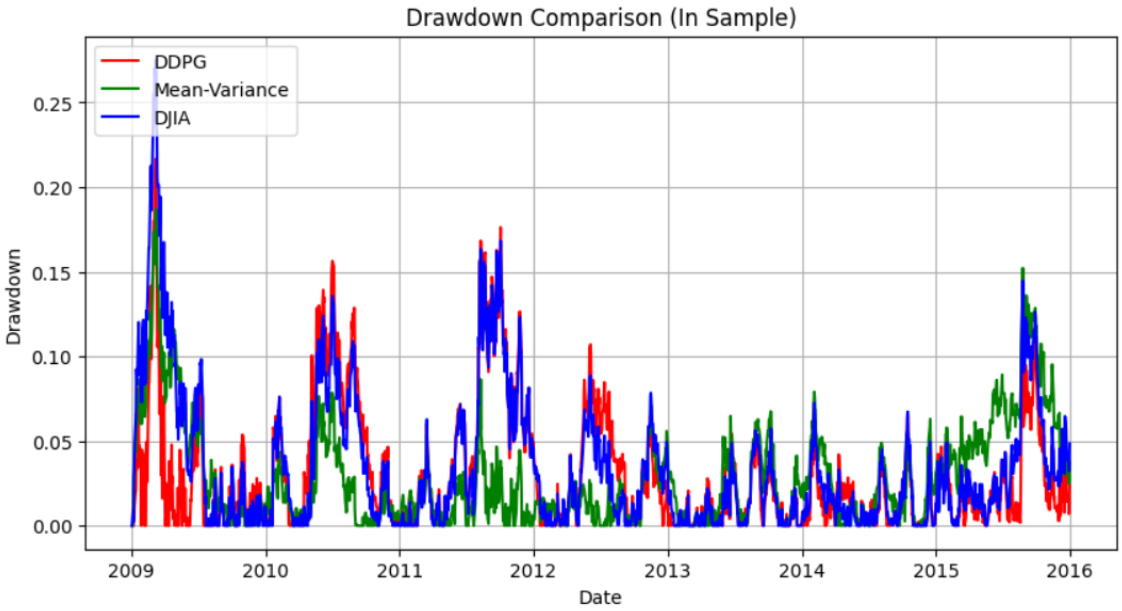


Figure 3: In-sample drawdown curves of our DDPG scheme, the mean-variance portfolio allocation strategy, and the Dow Jones Industrial Average (DJIA), with an initial portfolio value of $10,000.

5.2 Out-of-sample result

Table 2 presents the in-sample performance comparison. The results show that the DDPG strategy outperforms the other two benchmark strategies across the main performance measures.

Table 2: Out of Sample Performance Comparison

	DDPG (ours)	Mean-Variance	DJIA
Initial Portfolio Value	10000.000000	10000.000000	10000.000000
Final Portfolio Value	16850.230810	14382.960858	15428.540417
Annualized Return	0.198654	0.141957	0.165302
Annualized Std. Error	0.126422	0.104968	0.117072
Sharpe Ratio	1.452704	1.209489	1.283835
Max Drawdown	0.106350	0.149980	0.115849
AVG Drawdown	0.021793	0.038329	0.023482
Sortino Ratio	572.786905	481.622443	495.405333

Figure 4 shows the DDPG strategy significantly outperforms Dow Jones Industrial Average and the min-variance portfolio allocation. While the DDPG strategy achieves substantially higher returns in the in-sample period, its out-of-sample performance is much closer to the benchmark strategies. This reduction in performance gap suggests that part of the strong in-sample results may be attributed to overfitting to historical data. In particular, although DDPG still maintains slightly higher returns and risk-adjusted performance, the advantage is less pronounced compared to the in-sample results.

We also observe that the DDPG strategy produces different results each time the code is executed. This is due to the stochastic nature of the training process, including random initialization of network parameters, random minibatch sampling from the replay buffer, and exploration noise added to the actor policy. As a result, the learned policy is not exactly the same across different runs, which leads to variation in portfolio performance.

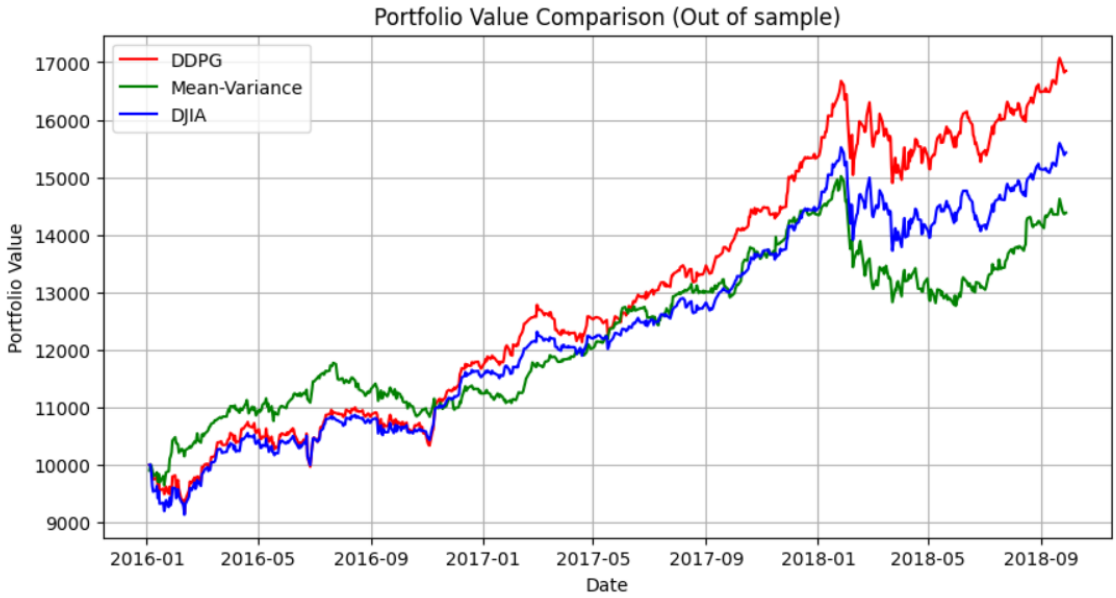


Figure 4: Out-of-sample portfolio value curves of our DDPG scheme, the mean-variance portfolio allocation strategy, and the Dow Jones Industrial Average (DJIA), with an initial portfolio value of $10,000.

Figure 5 illustrates the drawdown dynamics of the DDPG strategy, the mean-variance portfolio, and the DJIA benchmark for the out-of-sample periods. The mean-variance strategy exhibits a sustained and relatively high drawdown during certain intervals, particularly around 2017–2018, indicating weaker adaptability to changing market conditions. In comparison, the DDPG strategy maintains lower and more controlled drawdowns over most of the period, with smaller and shorter-lived spikes. The DJIA shows moderate drawdowns, generally lying between the other two strategies.

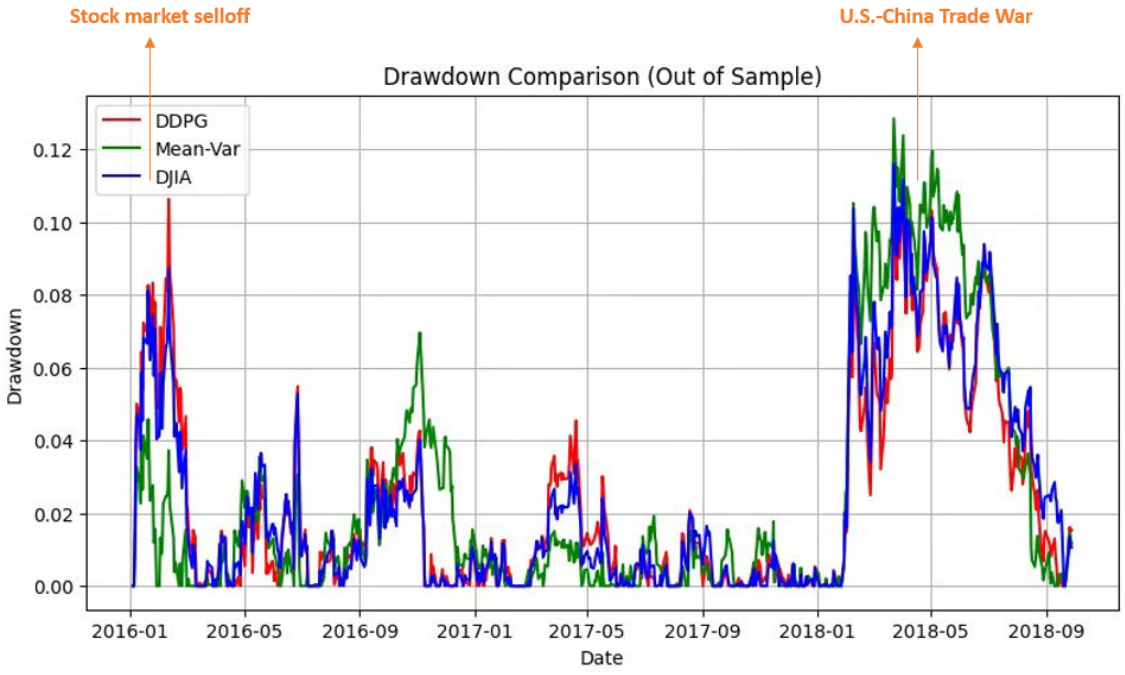


Figure 5: Out-of-sample drawdown curves of our DDPG scheme, the mean-variance portfolio allocation strategy, and the Dow Jones Industrial Average (DJIA), with an initial portfolio value of $10,000.

6 Conclusion

This report investigated the application of the Deep Deterministic Policy Gradient (DDPG) algorithm to portfolio optimization across the 30 constituent stocks of the Dow Jones Industrial Average. The proposed approach was evaluated against two benchmark strategies: the mean-variance portfolio and the passive DJIA index, using a range of performance metrics including annualized return, Sharpe ratio, maximum drawdown, and Sortino ratio.

The empirical results demonstrate that the DDPG-based strategy achieves competitive performance relative to both benchmarks. In the in-sample period, the DDPG portfolio attains a higher final portfolio value, annualized return, and Sharpe ratio than either the mean-variance portfolio or the DJIA index, while also maintaining a lower average drawdown. In the out-of-sample period, the DDPG strategy continues to outperform both benchmarks across most metrics, achieving the highest annualized return, Sharpe ratio, and Sortino ratio, as well as the lowest maximum and average drawdowns.

However, several limitations of the approach were observed. The performance gap between the in-sample and out-of-sample results suggests that the model may be prone to overfitting to historical market patterns. Furthermore, the stochastic nature of the training process, arising from random network initialization, random minibatch sampling, and exploration noise, leads to variability in outcomes across different runs. This sensitivity to hyperparameters and random seeds makes it difficult to reliably replicate strong results, highlighting the instability that is inherent to deep reinforcement learning methods in financial applications.

Overall, the results suggest that while DDPG holds promise as a data-driven approach to dynamic portfolio allocation, its practical deployment requires careful attention to training stability and generalization. Future work could explore more robust training procedures, ensemble methods across

multiple random seeds, or regularization techniques to reduce overfitting and improve out-of-sample consistency.

References

[1] Alexei Chekhlov, Stanislav P. Uryasev, and Michael Zabarankin. Portfolio optimization with drawdown constraints. *SSRN Electronic Journal*, 2000.

[2] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[3] Xiao-Yang Liu, Zhuoran Xiong, Shan Zhong, Hongyang (Bruce) Yang, and Anwar Walid. Practical deep reinforcement learning approach for stock trading. *arXiv preprint arXiv:1811.07522*, 2022. arXiv:1811.07522v3 [cs.LG], July 30, 2022.

[4] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

[5] William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1):119–138, 1966.

[6] Wikipedia contributors. 2015–2016 stock market selloff. `https://en.wikipedia.org/wiki/2015%E2%80%932016_stock_market_selloff`, 2026. Accessed: 2026-05-05.

[7] Wikipedia contributors. China–united states trade war. `https://en.wikipedia.org/wiki/China%E2%80%93United_States_trade_war`, 2026. Accessed: 2026-05-05.